

1. Write a C program to perform realization of Singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

/* Function to allocate memory for a node */
struct Node* getNode() {
    return (struct Node*)malloc(sizeof(struct Node));
}

void insertAtBeginning(int value) {
    struct Node *newNode = getNode();
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

void insertAtEnd(int value) {
    struct Node *newNode = getNode();
    struct Node *temp = head;

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

void deleteFromBeginning() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
}
```

```

    struct Node *temp = head;
    head = head->next;
    free(temp);
}

void deleteFromEnd() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    if (head->next == NULL) {
        free(head);
        head = NULL;
        return;
    }

    struct Node *temp = head;
    while (temp->next->next != NULL)
        temp = temp->next;

    free(temp->next);
    temp->next = NULL;
}

void display() {
    struct Node *temp = head;

    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n1.Insert at Beginning");
        printf("\n2.Insert at End");
        printf("\n3.Delete from Beginning");
    }
}

```

```

printf("\n4.Delete from End");
printf("\n5.Display");
printf("\n6.Exit\n");

scanf("%d", &choice);

switch (choice) {
    case 1:
        scanf("%d", &value);
        insertAtBeginning(value);
        break;

    case 2:
        scanf("%d", &value);
        insertAtEnd(value);
        break;

    case 3:
        deleteFromBeginning();
        break;

    case 4:
        deleteFromEnd();
        break;

    case 5:
        display();
        break;

    case 6:
        printf("Program Terminated\n");
        exit(0);
}
}
return 0;
}

```

Output:

- 1.Insert at Beginning
- 2.Insert at End
- 3.Delete from Beginning
- 4.Delete from End
- 5.Display
- 6.Exit

```
1
10
2
20
2
30

5
10 20 30

4

5
10 20

3

5
20

6
Program Terminated
```

2. Write a C program to perform realization of Doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

typedef struct Node* NodePtr;

NodePtr head = NULL;

/* getNode with direct return (no if, no variable) */
NodePtr getNode() {
    return (NodePtr)malloc(sizeof(struct Node));
}

void insertAtBeginning(int value) {
```

```

NodePtr newNode = getNode();

newNode->data = value;
newNode->prev = NULL;
newNode->next = head;

if (head != NULL)
    head->prev = newNode;

head = newNode;
}

```

```

void insertAtEnd(int value) {
    NodePtr newNode = getNode();
    NodePtr temp = head;

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        return;
    }

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

```

```

void deleteFromBeginning() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
}

```

```

NodePtr temp = head;
head = head->next;

if (head != NULL)
    head->prev = NULL;

free(temp);
}

```

```

void deleteFromEnd() {

```

```

if (head == NULL) {
    printf("List is empty\n");
    return;
}

if (head->next == NULL) {
    free(head);
    head = NULL;
    return;
}

NodePtr temp = head;
while (temp->next != NULL)
    temp = temp->next;

temp->prev->next = NULL;
free(temp);
}

void display() {
    NodePtr temp = head;

    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n1.Insert at Beginning");
        printf("\n2.Insert at End");
        printf("\n3.Delete from Beginning");
        printf("\n4.Delete from End");
        printf("\n5.Display");
        printf("\n6.Exit\n");

        scanf("%d", &choice);

        switch (choice) {

```

```

        case 1:
            scanf("%d", &value);
            insertAtBeginning(value);
            break;

        case 2:
            scanf("%d", &value);
            insertAtEnd(value);
            break;

        case 3:
            deleteFromBeginning();
            break;

        case 4:
            deleteFromEnd();
            break;

        case 5:
            display();
            break;

        case 6:
            printf("Program Terminated\n");
            exit(0);
    }
}
return 0;
}

```

Output:

```

1.Insert at Beginning
2.Insert at End
3.Delete from Beginning
4.Delete from End
5.Display
6.Exit
1
10
2
20
2
30
5
10 20 30

```

```
4
5
10 20

3
5
20

6
Program Terminated
```

3. Write a C program to perform realization of Circular Singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

typedef struct Node* NodePtr;

NodePtr tail = NULL;

/* getNode with direct return, no if, no extra variable */
NodePtr getNode() {
    return (NodePtr)malloc(sizeof(struct Node));
}

void insertAtBeginning(int value) {
    NodePtr newNode = getNode();
    newNode->data = value;

    if (tail == NULL) {
        tail = newNode;
        tail->next = tail;
    } else {
        newNode->next = tail->next;
        tail->next = newNode;
    }
}

void insertAtEnd(int value) {
    NodePtr newNode = getNode();
```



```

newNode->data = value;

if (tail == NULL) {
    tail = newNode;
    tail->next = tail;
} else {
    newNode->next = tail->next;
    tail->next = newNode;
    tail = newNode;
}
}

```

```

void deleteFromBeginning() {
    if (tail == NULL)
        return;

    NodePtr head = tail->next;

    if (tail == head) {
        free(tail);
        tail = NULL;
    } else {
        tail->next = head->next;
        free(head);
    }
}

```

```

void deleteFromEnd() {
    if (tail == NULL)
        return;

    NodePtr head = tail->next;

    if (head == tail) {
        free(tail);
        tail = NULL;
    } else {
        NodePtr temp = head;
        while (temp->next != tail)
            temp = temp->next;

        temp->next = head;
        free(tail);
        tail = temp;
    }
}

```

```

void display() {

```

```

if (tail == NULL) {
    printf("List is empty\n");
    return;
}

NodePtr temp = tail->next;
do {
    printf("%d ", temp->data);
    temp = temp->next;
} while (temp != tail->next);
printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n1.Insert at Beginning");
        printf("\n2.Insert at End");
        printf("\n3.Delete from Beginning");
        printf("\n4.Delete from End");
        printf("\n5.Display");
        printf("\n6.Exit\n");

        scanf("%d", &choice);

        switch (choice) {
            case 1:
                scanf("%d", &value);
                insertAtBeginning(value);
                break;

            case 2:
                scanf("%d", &value);
                insertAtEnd(value);
                break;

            case 3:
                deleteFromBeginning();
                break;

            case 4:
                deleteFromEnd();
                break;

            case 5:
                display();
                break;

```

```

        case 6:
            printf("Program Terminated\n");
            exit(0);
        }
    }
    return 0;
}

```

Output:

```

1.Insert at Beginning
2.Insert at End
3.Delete from Beginning
4.Delete from End
5.Display
6.Exit
1
10
2
20
2
30

5
10 20 30

4

5
10 20

3

5
20

6
Program Terminated

```

4. Write a C program to perform realization of Circular Doubly linked list.

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Node {

```

```

    int data;
    struct Node *prev;
    struct Node *next;
};

typedef struct Node* NodePtr;

NodePtr head = NULL;

/* getNode with direct return */
NodePtr getNode() {
    return (NodePtr)malloc(sizeof(struct Node));
}

void insertAtBeginning(int value) {
    NodePtr newNode = getNode();
    newNode->data = value;

    if (head == NULL) {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
    } else {
        NodePtr last = head->prev;

        newNode->next = head;
        newNode->prev = last;

        head->prev = newNode;
        last->next = newNode;

        head = newNode;
    }
}

void insertAtEnd(int value) {
    NodePtr newNode = getNode();
    newNode->data = value;

    if (head == NULL) {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
    } else {
        NodePtr last = head->prev;

        newNode->next = head;
        newNode->prev = last;
    }
}

```

```

        last->next = newNode;
        head->prev = newNode;
    }
}

void deleteFromBeginning() {
    if (head == NULL)
        return;

    if (head->next == head) {
        free(head);
        head = NULL;
    } else {
        NodePtr last = head->prev;
        NodePtr temp = head;

        head = head->next;
        head->prev = last;
        last->next = head;

        free(temp);
    }
}

void deleteFromEnd() {
    if (head == NULL)
        return;

    if (head->next == head) {
        free(head);
        head = NULL;
    } else {
        NodePtr last = head->prev;
        NodePtr beforeLast = last->prev;

        beforeLast->next = head;
        head->prev = beforeLast;

        free(last);
    }
}

void display() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
}

```

```

NodePtr temp = head;
do {
    printf("%d ", temp->data);
    temp = temp->next;
} while (temp != head);

printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n1.Insert at Beginning");
        printf("\n2.Insert at End");
        printf("\n3.Delete from Beginning");
        printf("\n4.Delete from End");
        printf("\n5.Display");
        printf("\n6.Exit\n");

        scanf("%d", &choice);

        switch (choice) {
            case 1:
                scanf("%d", &value);
                insertAtBeginning(value);
                break;

            case 2:
                scanf("%d", &value);
                insertAtEnd(value);
                break;

            case 3:
                deleteFromBeginning();
                break;

            case 4:
                deleteFromEnd();
                break;

            case 5:
                display();
                break;

            case 6:
                printf("Program Terminated\n");

```

```
        exit(0);
    }
}
return 0;
}
```

Output:

```
1.Insert at Beginning
2.Insert at End
3.Delete from Beginning
4.Delete from End
5.Display
6.Exit
1
10
2
20
2
30

5
10 20 30

4

5
10 20

3

5
20

6
Program Terminated
```